

LLD Case Study 3: Movie Ticket Booking System (BookMyShow / Fandango)

Target Patterns: Concurrency Control, State Machine, Factory Pattern, Strategy Pattern

Key Engineering Focus: High-concurrency seat locking, race condition elimination, temporary seat reservations with TTL expiration, and payment processing.

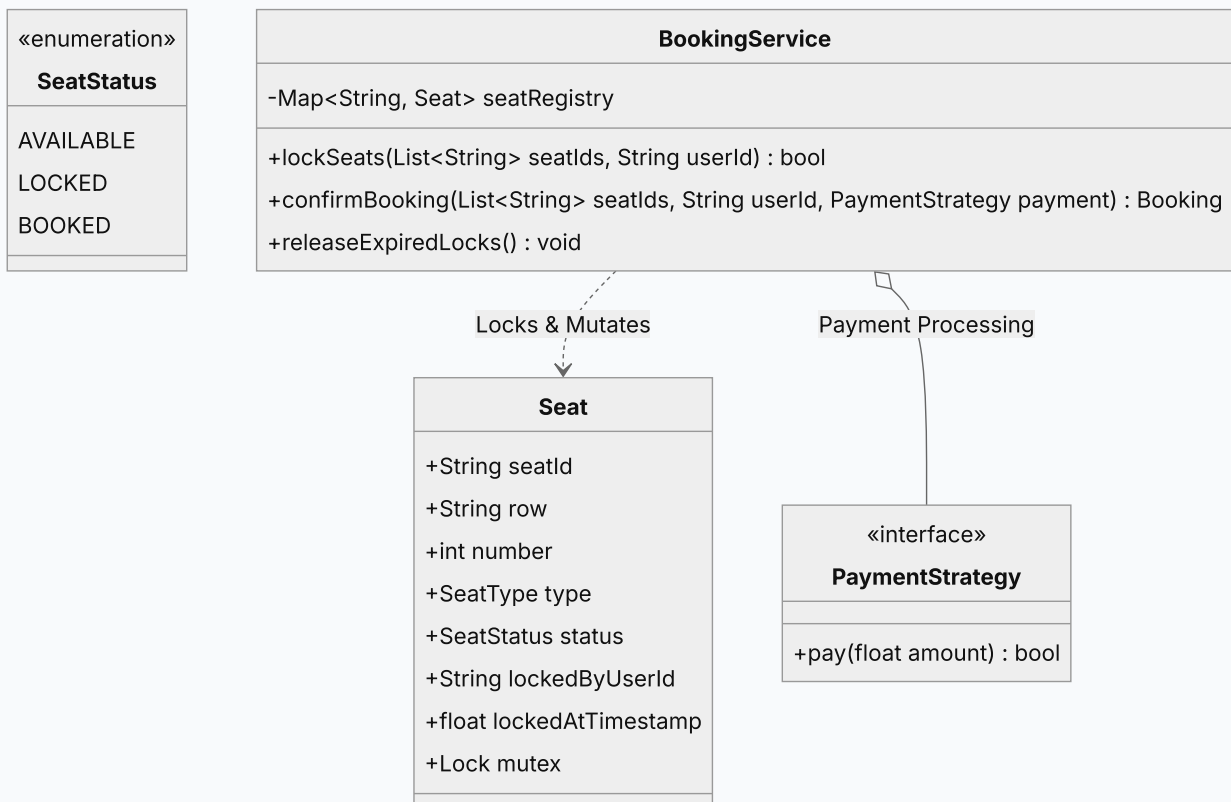
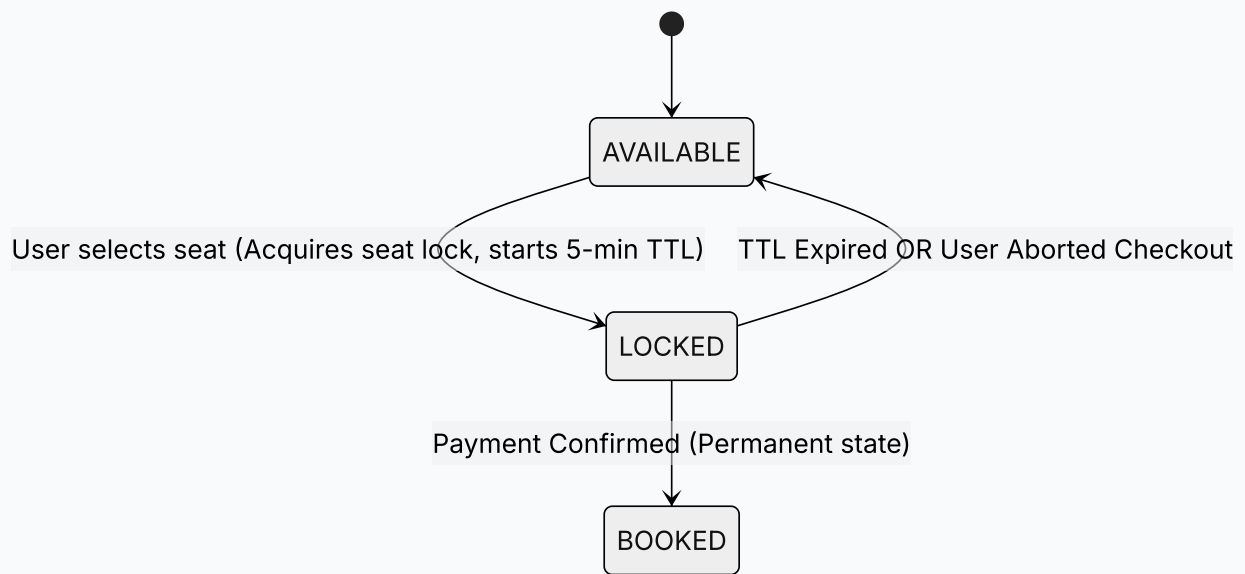
1. Problem Statement & Functional Requirements

Design the core booking engine for a cinema platform. Thousands of concurrent users vie for high-demand premiere seats.

Requirements:

1. **Cinema Hierarchy:** Cinema → Audi / Screen → Show → Seats (Regular, VIP, Recliner).
 2. **State Machine:** A seat can be `AVAILABLE`, `TEMPORARILY_LOCKED` (e.g., 5-minute payment hold window), or `BOOKED`.
 3. **Concurrency Control:** Absolutely no two users can book or lock the same seat simultaneously. Handled via thread-safe atomic mutexes.
 4. **Auto-Expiration:** If payment is not completed within the hold window, the lock automatically expires and the seat returns to `AVAILABLE`.
 5. **Payment Processing:** Decoupled via Strategy Pattern (UPI, CreditCard, NetBanking).
-

2. Architecture & State Diagram



3. Production-Grade Python Implementation

4. Edge Cases, Tests and Extensions

What the design guarantees and where it needs help

Concern	Status	Detail
Two users, one seat	Safe	Per-seat locks plus a status check inside the lock: only one <code>lock_seats</code> call succeeds
Deadlock across multi-seat requests	Safe	Seat ids are sorted before locking, so every thread acquires locks in the same global order
Abandoned checkout	Safe	A lock older than <code>LOCK_TTL_SECONDS</code> is treated as free by the next user
Unknown seat id	Safe	<code>KeyError</code> is raised and the <code>finally</code> block releases the locks acquired so far
Payment fails	By design	Seats stay <code>LOCKED</code> so the user can retry until the TTL runs out
Payment called while holding seat locks	Weak	A slow gateway blocks every other request touching those seats; real systems release locks, call the gateway, then re-verify
Charged but not booked	Not handled	A crash between <code>process_payment</code> and the status change loses the booking; use an idempotency key and reconcile

Tests

This block extends the implementation above (clock patched for the TTL tests).

Extensions interviewers ask for

1. **Shows, screens and theatres:** seats belong to a `Show` (a movie at a time on a screen), not to a theatre; the same physical seat is a different bookable object per show.
2. **Seat maps and pricing tiers:** price comes from a `PricingStrategy` using seat class, show time and demand, evaluated when the lock is taken and stored with the lock.

3. **Distributed deployment:** move seat state to a database; the lock becomes `UPDATE seat SET status='LOCKED', locked_by=?, locked_until=? WHERE id=? AND (status='AVAILABLE' OR locked_until < now())` and the affected row count says who won.
4. **Waiting room under flash-sale load:** put a queue in front of the lock endpoint so a popular release does not hammer the seat table.
5. **Refunds and cancellation:** a `BOOKED → CANCELLED → AVAILABLE` path with a refund call that is idempotent.

Follow-up questions

- *Why lock seats in sorted order?* A global ordering of lock acquisition removes circular waits, one of the four conditions for deadlock.
- *Why a TTL instead of waiting for the user to cancel?* Users abandon checkouts silently; without expiry, popular seats would stay blocked forever.
- *How do you avoid charging twice on retry?* Send an idempotency key (booking id) to the payment provider so a repeated charge returns the first result.